



9. Patrones de comunicación entre gameObjects

Física e Inteligencia Artificial para Videojuegos

Doble Grado en Física e Inteligencia Artificial

Curso 2025 – 2026

Fernando Madrid Recio

Referencia directa: Uso de [SerializeField]

- Permite asignar referencias directamente desde el Inspector de Unity manteniendo las variables privadas.
- Uso aconsejado: Principalmente para dependencias internas dentro de un mismo Prefab.
- Beneficio arquitectónico: Mantiene una alta cohesión y evita que el Prefab dependa de la jerarquía específica de una escena, facilitando su reutilización.

```
// Mantiene la variable protegida pero accesible en el Inspector  
[SerializeField] private GameObject weapon;  
[SerializeField] private Transform spawnPoint;
```

Referencia por búsqueda: Métodos Find

- Métodos como `GameObject.Find()`, `FindObjectOfType()` o `FindWithTag()`.
- Impacto en Rendimiento: Operaciones extremadamente costosas. Obligan a la CPU a iterar sobre toda la jerarquía de la escena actual.
- Escalabilidad: Generan código frágil (dependen de strings o nombres específicos que pueden cambiar).
- Regla de oro: Solo deben usarse si no hay otra opción de diseño, y estrictamente durante la fase de inicialización (`Awake` o `Start`), nunca en `Update`.

```
void Start()
{
    // Evitar esto en la medida de lo posible
    player = GameObject.Find("PlayerCharacter");
}
```


Referencia en interacciones físicas

- Resolución de dependencias en tiempo real mediante el motor de físicas: OnTriggerEnter, OnCollisionEnter y Raycast.
- El objetivo es obtener dinámicamente el GameObject con el que se ha colisionado.
- Comunicación Segura: Se debe priorizar el uso de TryGetComponent sobre GetComponent. Esto evita excepciones de referencia nula (NullReferenceException) en un solo paso optimizado.

```
private void OnTriggerEnter(Collider other)
{
    // Obtención segura: verifica y asigna la referencia simultáneamente
    if (other.TryGetComponent<Player>(out Player player))
    {
        player.TakeDamage(10);
    }
}
```

Encapsulamiento al usar variables de otras clases

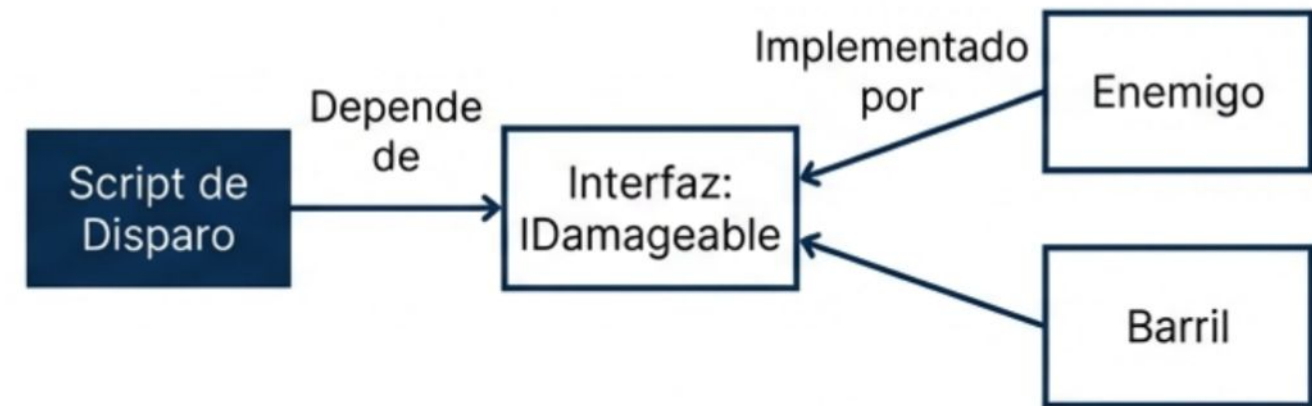
- Exponer variables públicas (public int health;) rompe el encapsulamiento y permite modificaciones incontroladas desde otros scripts.
- Solución C#: Uso de Auto-properties (getters y setters).
- Permiten lectura pública mientras restringen la escritura (modificación) únicamente a la clase propietaria.
- Garantiza que el estado interno del objeto interactúe de forma segura y predecible.

```
// Lectura pública, escritura estrictamente privada
public int Health { get; private set; }

public void TakeDamage(int damage)
{
    Health -= damage; // Solo esta clase puede modificar la variable
}
```

Desacoplamiento a partir del uso de Interfaces

- En lugar de buscar un componente específico (ej. EnemyScript), buscamos un contrato de comportamiento.
- Reduce drásticamente la dependencia directa entre clases (**Tight Coupling**).
- Permite que distintos objetos (jugador, barril explosivo, enemigo) respondan a la **misma interacción** sin compartir el mismo código base.



```

public interface IDamageable
{
    void ApplyDamage(int amount);
}

// Uso combinado con TryGetComponent
if (other.TryGetComponent<IDamageable>(out IDamageable target)) {
    target.ApplyDamage(10);
}
  
```


Desacoplamiento con patrón Singleton

- Patrón de diseño estructural que garantiza que solo exista una instancia de una clase con un punto de acceso global.
- Beneficios: Ideal para Managers (GameManager, AudioManager, UIManager). Elimina la necesidad de pasar referencias repetitivamente a través de múltiples scripts.
- Precaución: Su abuso puede generar un estado global inmanejable y ocultar dependencias (Anti-patrón si se usa para todo).

```
public class GameManager : MonoBehaviour
{
     2 usages
    public static GameManager Instance { get; private set; }

     Event function
    private void Awake()
    {
        if (Instance ==  null)
        {
            Instance = this;
        }
        else
        {
            Destroy(gameObject);
        }
    }
}
```

Tipos de Singletons

```
public class GameManager : MonoBehaviour
{
    2 usages
    public static GameManager Instance { get; private set; }

    Event function
    private void Awake()
    {
        if (Instance == null)
        {
            Instance = this;
            DontDestroyOnLoad(target: this);
        }
        else
        {
            Destroy(gameObject);
        }
    }
}
```

- Se puede distinguir entre **Singletons** de escena (sólo existen en la escena en la que se crean) y Singletons de juego (persistentes a toda la sesión de juego, pasan de escena en escena).
- Para poder hacer un Singleton de juego, basta con añadir el método **DontDestroyOnLoad()**